

08 · Flujo de datos

Documentacion del sistema

Version analizada: **0.3.0**

Commit analizado: **ff246ab**

Fecha de generacion: **2026-08-27**

Generado a partir de 08-data-flow.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Panorama: las cinco fuentes y los cuatro destinos
2. Origen de los datos
3. Validación: qué se comprueba y qué no
4. Transformación: el ciclo de un dato dentro de una corrida
5. Almacenamiento: qué acaba dónde
6. Consumo: quién lee qué
7. Dónde se pierden o corrompen datos
8. Datos personales y sensibles
9. Qué sale del equipo
10. Tres flujos completos, de punta a punta
11. Integridad y determinismo

De dónde vienen los datos, cómo se validan, cómo se transforman, dónde se guardan, quién los consume, dónde se pueden perder y qué datos personales o sensibles atraviesan el sistema.

1. Panorama: las cinco fuentes y los cuatro destinos

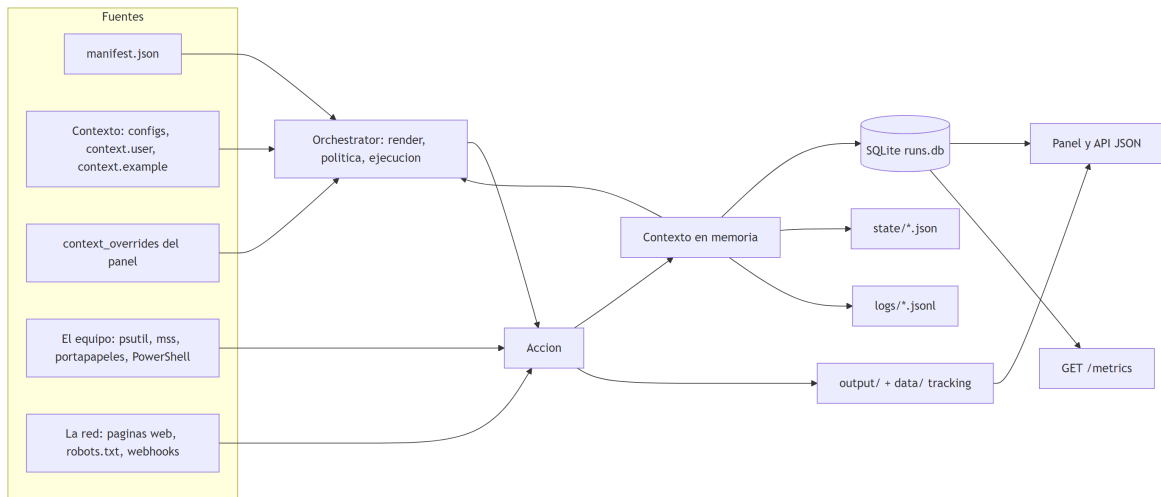


Diagrama 1 (Mermaid, renderizado)

Lo que el diagrama muestra: las tres fuentes de configuración entran por el motor y las dos fuentes de mundo real entran por la acción. Todo converge en el contexto, y el contexto se persiste por triplicado.

Lo que no muestra: que el contexto es **acumulativo y una sola referencia**. No hay copia por paso: `self.state['context'] = self.context` apunta al mismo diccionario. Lo que un paso guarda con `save_as` queda visible para todos los siguientes y acaba entero en la columna `context_json` de la tabla `runs`.

2. Origen de los datos

2.1 El manifest — datos de estructura

`flows/<carpeta>/manifest.json`, leído por `FlowLoader.load_manifest`. Define pasos, acciones, parámetros con placeholders, condiciones y transiciones. **No se valida contra el JSON Schema en tiempo de ejecución:** el schema solo lo aplica `scripts/validate_project.py` en la CI. Un manifest inválido llega hasta el motor y falla con `KeyError`.

2.2 El contexto — datos de configuración

`FlowLoader.load_context` resuelve la **primera fuente que exista** y descarta las demás:

Prioridad	Fuente	Quién la escribe
1	<code>--context RUTA</code> del CLI	El operador
2	<code>configs/<carpeta>.json</code>	El panel, en <code>POST /flow/<f>/config</code>
3	<code>flows/<carpeta>/context.user.json</code>	El operador, a mano. No versionado
4	<code>flows/<carpeta>/context.example.json</code>	El autor del flow. Versionado

No hay mezcla de claves. Si existe `configs/03_folder_inventory.json`, el `context.example.json` del flow 03 se ignora por completo, aunque una versión posterior del flow añada claves nuevas. Es la trampa de configuración más probable del sistema.

2.3 Los overrides — datos de invocación

```
POST /api/run/03_folder_inventory
{"context_overrides": {"path_override": "C:/Users/ejemplo/Documentos"}}
```

Se fusionan con `{**contexto, **overrides}`: **reemplazo superficial de claves de primer nivel**. Un override de un objeto anidado borra el resto de sus claves.

2.4 El equipo — datos del mundo real

Dato	Acción	Biblioteca
CPU, RAM, disco	<code>system.snapshot_system</code>	<code>psutil</code>
Procesos	<code>system.top_processes</code>	<code>psutil</code>
Píxeles del escritorio	<code>screen.capture_*</code>	<code>mss</code> , respaldo <code>Pillow</code>
Ventana en foco	<code>screen.capture_active_window</code>	<code>PyGetWindow</code>
Portapapeles	<code>system.read_clipboard</code>	<code>pyperclip</code>
Inventario del sistema	<code>system.run_powershell</code>	<code>subprocess</code> + PowerShell
Texto en imágenes	<code>vision.ocr_image</code>	<code>pytesseract</code> + binario <code>tesseract</code>
Archivos y carpetas	<code>filesystem.*</code>	Biblioteca estándar

2.5 La red — datos externos

Dato	Acción	Salida
DOM renderizado	<code>browser.extract_content</code> , <code>crawl_site</code> , <code>capture_page</code>	Chromium
Estado de enlaces	<code>http.check_urls</code>	<code>requests</code> HEAD/GET
Contenido de una URL	<code>http.fetch_url</code>	<code>requests</code> GET
<code>robots.txt</code>	<code>RobotsCache._fetch</code>	<code>requests</code> GET
Notificación	<code>notify.send</code> con backend: <code>webhook</code>	<code>requests</code> POST

Todos los flows web apuntan por defecto a archivos locales del repositorio (`data/web/demo_page.html`, `data/web/site_demo/index.html`, `data/web/control_page.html`). Verificado leyendo los ocho `context.example.json` correspondientes. Sin cambiar la configuración, **el sistema no genera tráfico de red**.

3. Validación: qué se comprueba y qué no

Punto	Qué valida	Cuándo	Módulo
JSON Schema	Estructura completa del manifest, <code>additionalProperties: false</code>	Solo en CI	<code>scripts/validate_project.py</code>
Acciones registradas	Que cada <code>action</code> exista	Solo en CI	<code>validate_project.py</code>
Coherencia con <code>allowed_actions</code>	Que ningún paso use una acción que su propia política prohíbe	Solo en CI	<code>validate_project.py</code>
Transiciones y <code>start_step</code>	Que apunten a pasos existentes	Solo en CI	<code>validate_project.py</code>

<code>_safe_folder</code>	Slug de la URL: <code>^[A-Za-z0-9_\-]{1,64}\$</code>	En cada petición	<code>app/server.py</code>
Ruta de <code>/file</code>	NUL, controles, absolutas, prefijo, extensión	En cada petición	<code>app/server.py::do_GET</code>
<code>_authorize_mutation</code>	Token, o <code>Host/Origin/Referer</code>	En cada POST	<code>app/server.py</code>
<code>assert_secrets_present</code>	Variables de entorno presentes y no vacías	Una vez por corrida	<code>engine/sandbox.py</code>
<code>assert_action_allowed</code>	Acción en la lista blanca	Antes de cada paso	<code>engine/sandbox.py</code>
<code>assert_paths_allowed</code>	Rutas renderizadas bajo <code>allowed_paths</code>	Antes de cada paso	<code>engine/sandbox.py</code>
Tokens de PowerShell	<code>;</code> , <code>\ </code> , <code>&</code> , backtick, <code>></code> , <code><</code> , <code>\$</code> (, <code>\$_</code>	En la acción	<code>actions/system.py</code>
Allowlist de verbos	Primera palabra del comando	En la acción	<code>actions/system.py</code>
<code>shell=True</code> prohibido	Cierra CWE-78	En la acción	<code>actions/ui.py</code>

Lo que NO se valida en ningún punto:

- Que los `params` de un paso coincidan con la **firma** de la acción. Un `screen.capture_screenshot` sin `output_path` pasa la CI y falla en ejecución con `TypeError`.
- Que el **tipo** de un valor del contexto sea el esperado. Un umbral escrito como `"80"` en vez de `80` produce `TypeError` al comparar en `conditions.matches`.
- El **contenido** de un `context_overrides`: se acepta cualquier diccionario.
- La **URL** de `browser.*: _to_url` solo comprueba que, si no es URL, el archivo exista.

4. Transformación: el ciclo de un dato dentro de una corrida

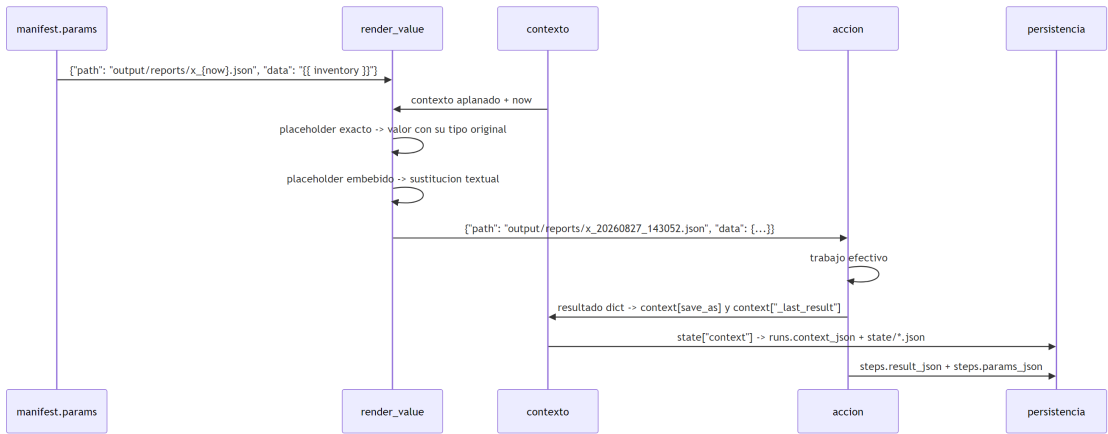


Diagrama 2 (Mermaid, renderizado)

Lo que la secuencia muestra: los dos modos de `render_value`. Un `"{{ inventory }}"` solo (placeholder exacto) devuelve el **diccionario entero**; un `"x_{now}.json"` (embebido) hace sustitución textual. Es lo que permite que el flow 04 pase el resultado completo de un paso como `data` del siguiente.

Lo que no muestra: que `{now}` se recalcula en **cada** llamada a `render_value`, es decir, una vez por paso. Dos pasos del mismo flow pueden obtener timestamps distintos si cruzan el cambio de segundo. Los flows del repositorio lo evitan pasando la ruta por el contexto con `save_as` en lugar de recomputar `{now}`.

Ejemplo real y completo: `04_document_drop_pipeline`

Cuatro pasos, encadenados por contexto. Rastro del dato paso a paso:

Paso	Acción	Entrada	save_as	Qué añade al contexto
scan_dropbox	filesystem.list_directory	{{ dropbox_path }}	inventory	{path, files[], total_files}
classify_dropbox	filesystem.classify_file	{{ inventory.files }}	stats	{total_files, total_size_bytes, by_extension, largest_file}
summarize_texts	filesystem.summarize_text	{{ dropbox_path }}, max_files: 10, max_chars_per_file: 500	summary	{path, processed_files, summaries[]}
write_pipeline_report	filesystem.write_json	{{ inventory }}, {{ stats }}, {{ summary }}	report	{path, written}

Nótese `{{ inventory.files }}` en el paso 2: usa el **contexto aplanado** para bajar un nivel dentro del resultado del paso 1. Es el mecanismo que hace innecesario cualquier código de pagamento.

Pérdida de información silenciosa en este flow: `summarize_text_folder` corta en `max_files: 10` y solo acepta cinco extensiones. Una carpeta con 50 logs produce un informe de 10, y el JSON **no dice cuántos quedaron fuera** — solo `processed_files: 10`. Es la única cota del sistema que no se declara explícitamente. Registrado en 15.

5. Almacenamiento: qué acaba dónde

Dato	runs.context_json	steps.result_json	state/*.json	logs/*.jsonl	output/
Parámetros renderizados	—	[OK] (params_json)	[OK]	[OK] (step_started)	—
Resultado de cada paso	[OK]	[OK]	[OK]	[OK] (step_finished)	—
Texto de una página web	[OK]	[OK]	[OK]	[OK]	[OK] si hay markdown
Portapapeles	[OK]	[OK]	[OK]	[OK]	[OK] JSON
Texto OCR del escritorio	[OK]	[OK]	[OK]	[OK]	[OK] JSON
Captura PNG	Solo la ruta	Solo la ruta	Solo la ruta	Solo la ruta	[OK] binario
Salida de PowerShell	[OK]	[OK]	[OK]	[OK]	[OK] JSON
Mensaje de error	[OK] (error_json)	[OK] (error_text)	[OK]	[OK] (step_failed)	—

Cuatro copias del mismo dato es la norma, no la excepción. El resultado de un paso se escribe en `steps.result_json`, en `runs.context_json`, en el snapshot de `state/` y en la línea `step_finished` del JSONL. Es redundancia deliberada por trazabilidad, y explica por qué el volumen crece rápido.

Tracking persistente entre corridas

Tres archivos JSON dan **memoria** a los flows más allá de una corrida:

Archivo	Flow	Qué recuerda	Qué pasa si se borra
---------	------	--------------	----------------------

<code>data/seeds/.used_indices.json</code>	07	IDs de registros ya usados	El ciclo empieza de cero; se repiten registros
<code>data/web_watch/demo_page.json</code>	23	SHA-256 del texto de la última corrida	Próxima corrida: <code>first_run: true</code> , no detecta el cambio
<code>data/web_watch/precio_demo.json</code>	26	Último valor leído del selector CSS	Igual

Ninguno está versionado. `data/web_watch/` ni siquiera existe en un clon limpio: se crea sola en la primera corrida. Es diseño, no defecto — la primera corrida establece la línea base y lo declara con `first_run: true`.

6. Consumo: quién lee qué

Consumidor	Fuente	Ruta
Pestaña Ejecutar	<code>flows</code> en disco + última corrida de <code>runs</code>	<code>render_home</code>
Pestaña Programadas	<code>schedules</code>	<code>render_home</code>
Pestaña Histórico	<code>runs</code>	<code>render_home</code> , <code>render_flow_history</code>
Detalle de corrida	<code>runs + steps + events</code>	<code>render_run_detail</code>
Polling durante la ejecución	<code>runs + steps</code> + manifest	<code>GET /api/runs/<run_id>/status</code>
Dashboard de métricas	Agregados de <code>runs + steps</code>	<code>render_metrics_dashboard</code>
Prometheus	Agregados	<code>GET /metrics</code>
Vistas previas e imágenes	<code>output/</code> vía <code>outputs_json</code>	<code>GET /file?path=...</code>
Integrador externo	<code>runs</code>	<code>GET /api/runs</code> , <code>GET /api/flows</code>

El resumen legible del panel

`app/server.py::_smart_summary` traduce el contexto crudo en texto para humanos. Pero solo cubre 6 de los 27 flows. Verificado leyendo la cadena de `elif` completa: hay rama para `screen_capture_analyze`, `screen_capture_browser`, `folder_inventory`, `document_drop_pipeline`, `system_healthcheck` y `process_watchdog`. Para los otros 21 —incluidos `browser_form_filler` y los siete de la familia web— la función devuelve cadena vacía y `render_run_detail` omite el bloque entero; el operador ve solo los pasos colapsables con su JSON.

Es una divergencia con el `README.md` del repositorio, que en su demo de cinco minutos afirma que el detalle del flow 07 muestra «los 10 datos enviados como lista legible». Registrado en 15 · Riesgos y deuda técnica.

Además, cada rama depende de las **claves concretas** que devuelve cada acción (`capture.width`, `stats.by_extension`, `snapshot.cpu_percent`...): cambiar el nombre de un campo de retorno degrada el resumen sin romper nada. Acoplamiento implícito entre presentación y acciones.

7. Dónde se pierden o corrompen datos

Punto	Qué se pierde	Se declara	Gravedad
<code>summarize_text_folder</code> con más de <code>max_files</code>	Los archivos sobrantes	[NO]	Media
<code>summarize_text_folder</code> con extensiones no soportadas	Los archivos ignorados	[NO]	Media

<code>top_processes</code> con <code>AccessDenied</code>	Procesos privilegiados	Parcial (<code>total_seen</code>)	Baja
<code>apply_tracking</code> con archivo de estado corrupto	La línea base	[NO] <code>Se reinicia en silencio</code>	Media
<code>resolve_links</code> con más de <code>max_links</code>	Enlaces sobrantes	[OK] <code>links_truncated</code>	Baja
<code>extract_content</code> con más de <code>max_text_chars</code>	Texto sobrante	[OK] <code>text_truncated</code>	Baja
<code>check_urls</code> con más de <code>max_urls</code>	URLs sobrantes	[OK] <code>truncated</code>	Baja
<code>run_powershell</code> con salida larga	Más allá de 50 000 / 5 000 chars	[OK] <code>*_truncated</code>	Baja
<code>read_clipboard</code> con más de <code>max_chars</code>	Texto sobrante	[OK] <code>truncated</code> + <code>length</code> real	Baja
<code>crawl_pages</code> con <code>max_pages</code> alcanzado	Páginas pendientes	[OK] <code>truncated</code>	Baja
<code>_pick_record</code> cuando el llenado falla después	Un registro del seed, marcado como usado sin llegar a enviarse	[NO]	Baja
Proceso muerto a mitad de corrida	Nada persistido; la fila queda en <code>running</code> para siempre	[NO]	Media
<code>set_flow_config</code> si falla el <code>INSERT</code> tras escribir el archivo	Desincronización archivo/tabla	[NO]	Baja
Dos corridas del mismo flow en paralelo desde el panel	Escritura concurrente del archivo de tracking	[NO]	Media

Balance honesto: el sistema declara nueve de sus trece cotas. Las cuatro que no lo hacen están concentradas en `summarize_text_folder` y en `apply_tracking`.

8. Datos personales y sensibles

Categoría	Flows	Dónde acaba	Retención
Contenido del portapapeles	15	<code>context_json</code> , <code>result_json</code> , <code>state/</code> , <code>output/reports/</code>	Indefinida
Capturas de pantalla	01, 09, 12, 16, 17, 19	<code>output/screenshots/*.png</code>	Indefinida
Texto OCR de ventanas abiertas	12, 17	<code>context_json</code> , <code>output/reports/</code>	Indefinida
Inventario del equipo	05, 06, 18	<code>context_json</code> , <code>output/reports/</code>	Indefinida
Rutas y nombres de archivo personales	03, 04, 10	<code>context_json</code> , <code>output/reports/</code>	Indefinida
Contenido de páginas web	21–27	<code>context_json</code> , <code>output/reports/</code> , <code>output/*.md</code>	Indefinida
Datos de formulario	07	<code>output/reports/form_submission/</code>	Indefinida. Sintéticos: vienen del seed

Los tres puntos de mayor exposición, en orden:

- 1. `system.read_clipboard` — el portapapeles puede contener contraseñas recién copiadas de un gestor. El flow 15 las escribiría en claro en cuatro sitios distintos.
- 2. **Las capturas de escritorio** — cualquier cosa visible: correo abierto, chat, un documento confidencial. Y `GET /file?path=...` sirve PNG sin autenticación: la allowlist de extensiones bloquea `.html`, `.js`, `.svg` y `.css`, pero no `.png`.
- 3. `GET /api/runs` — devuelve `context_json` de todas las corridas, sin exigir token, incluso con `AUTOMA_PANEL_TOKEN` definido (`do_GET` no llama a `_authorize_mutation`).

Ningún control de privacidad está implementado: no hay anonimización, no hay redacción de campos, no hay cifrado en reposo, no hay retención. Análisis completo en 11 · Seguridad.

9. Qué sale del equipo

Con la configuración por defecto: **nada**. Verificado leyendo los 27 `context.example.json` y los 27 manifests.

Salida potencial	Se activa cuando	Por defecto
Petición HTTP a una página	El operador cambia <code>target_url</code> a una URL remota	Apunta a <code>data/web/*.html</code>
<code>robots.txt</code>	Igual, en el flow 22	Local, sin red
Verificación de enlaces	Flow 24 sobre una página con enlaces externos	Página local con enlaces locales
Webhook de notificación	<code>notify_backend: "webhook"</code> en los flows 23 o 26	<code>"file"</code>
Proveedor de IA	Un flow nuevo que llame a <code>vision.inspect_screen_target</code> con proveedor <code>≠ mock</code>	Ningún flow lo usa
Telemetría del producto	—	No existe

El único servidor que Automa levanta escucha en `127.0.0.1`. No hay cliente que se conecte a ningún servicio del proyecto.

10. Tres flujos completos, de punta a punta

10.1 15_clipboard_capture — el más corto y el más sensible

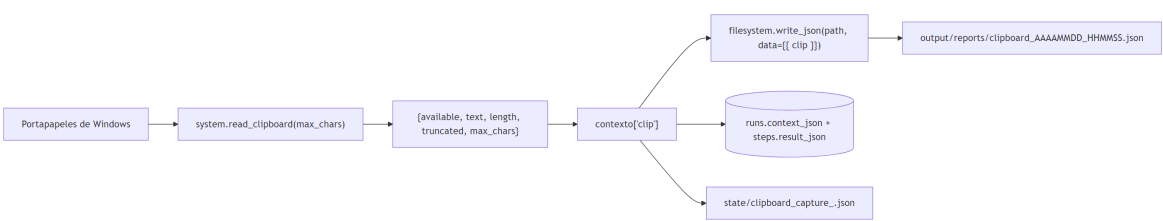


Diagrama 3 (Mermaid, renderizado)

Dos pasos, una lectura del sistema, **cuatro copias del texto copiado**. El flow declara `allowed_actions` y `allowed_paths: ["output/reports"]`, así que `assert_paths_allowed` comprueba la clave `path` del segundo paso. Pero la política **no impide** que el texto acabe en `context_json`: el sandbox controla dónde escriben las acciones, no qué guarda el motor.

10.2 23_web_change_detector — el que usa memoria entre corridas

Paso	Acción	Decisión
------	--------	----------

